Lecture 14: Unsupervised Learning: Dimensionality Reduction and Advanced Topics

Readings: ISL (Ch. 12), PRML[1] (Ch. 12), ESL (Ch. 14); code

Soon Hoe Lim

October 9, 2025

[1] In addition to ISL and ESL, we use Bishop's PRML as our reference for unsupervised learning, which provides more in-depth treatment of core methods with a unified probabilistic framework. ESL Ch. 14 covers a broader range of topics but with less depth on the fundamentals we emphasize. Also, note that Ch. 16 in a modern version of PRML covers similar material; see `https://www.bishopbook.com/`.

Outline

Unsupervised Learning: Overview

Given dataset $\{x_i\}_{i=1}^N$ where $x_i \in \mathbb{R}^d$, without labels, the main goal is to discover structure or learn properties of the underlying data distribution.

Key differences from supervised learning:

- Not interested in prediction: no labels/responses to guide learning
- More subjective - no simple goal for the analysis
- Success depends on interpretability and downstream utility
- Often used for exploratory data analysis: visualization and preprocessing

Main tasks in unsupervised learning:

1. **Clustering**: Partition data into K groups with similar characteristics
2. **Dimensionality Reduction**: Find low-dimensional representation $z_i \in \mathbb{R}^m$ ($m < d$) that retains key information
3. **Density Estimation**: Model distribution ρ^* from which $\{x_i\}$ are sampled
4. **Generative Modeling**: Build models that can produce new samples from learned distribution

Principal Component Analysis (PCA)

We will focus on dimensionality reduction methods today.

Given data $\{x_i\}_{i=1}^N$ where $x_i \in \mathbb{R}^d$, we want to find a transformation from $\mathbb{R}^d$ to $\mathbb{R}^m$ with $m \leq d$ (often much smaller).

PCA[2] refers to the process by which **principal components**[3] (PCs) are computed, and the subsequent use of these PCs in understanding the data.

Goal of PCA: Find a low-dimensional representation of the data that "capture as much of the information as possible" with **linear** transformation.

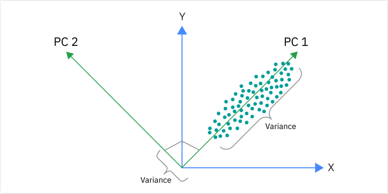
It is very useful not only for data visualization, but also for dimensional reduction and compression of data.

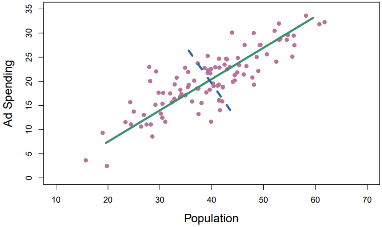
[2]PCA is one of the oldest methods for dim reduction, and has been rediscovered many times in many fields, so it is also known as the Karhunen-Loève transform, the Hotelling transform and proper orthogonal decomposition (POD).

[3]Principal components are briefly discussed in Lecture 2 in the context of principal components regression, which use PCs as predictors in place of the larger set of correlated variables.

PCA: Intuition and Visualization

Let's think about how to summarize a two-dimensional dataset in one dimension:





Intuition 1: PCA identifies directions in feature space along which the data exhibits the greatest variability. The first PC corresponds to the direction of maximum variance; the second captures the next largest variance, subject to being orthogonal to the first, and so on.

Intuition 2: PCA can be viewed as finding a low-dimensional subspace that best reconstructs (compress) the data. The first PC defines the one-dimensional subspace minimizing reconstruction error. The second principal component then adds the next direction—orthogonal to the first—that most reduces the remaining reconstruction error, and so on.

Review: Eigenvalues and Eigenvectors

Mathematically, PCA corresponds to an **eigendecomposition** of the sample covariance matrix. Before deriving this, let's recall some linear algebra facts.

For a square matrix $A \in \mathbb{R}^{d \times d}$:

1. **Eigenvector and eigenvalue**: A pair (u, λ) where $u \neq 0$ and $\lambda \in \mathbb{R}$ such that: $Au = \lambda u$
2. **Diagonalizable**: A is diagonalizable if there exists diagonal Λ and invertible P such that:
$$A = P \Lambda P^{-1}$$
3. **Symmetric**: A is symmetric if $A^T = A$
4. **Orthogonal**: A is orthogonal if $A^T A = A A^T = I$ (equivalently, $A^T = A^{-1}$)

Key Result: If A is symmetric, then A is diagonalizable by orthogonal matrices with real eigenvalues:
$$A = U \Lambda U^T$$
where U is orthogonal and $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_d)$ with real eigenvalues. The columns $\{u_j\}$ of U are orthonormal eigenvectors: $u_i^T u_j = \delta_{ij}$

Review: Positive (Semi-)Definite Matrices

Positive semi-definite (PSD): A symmetric matrix A is PSD if:

$$x^T A x \geq 0 \quad \text{for all } x \in \mathbb{R}^d$$

Equivalent characterization: A is PSD if and only if all eigenvalues are non-negative: $\lambda_i \geq 0$ for all i.

Positive definite (PD): A is PD if:

$$x^T A x > 0 \quad \text{for all } x \neq 0$$

Equivalent characterization: A is PD if and only if all eigenvalues are strictly positive: $\lambda_i > 0$ for all i.

Convention: For a PSD matrix with eigendecomposition $A = U \Lambda U^T$ where $\Lambda = \text{diag}(\lambda_1, \ldots, \lambda_d)$, we arrange eigenvalues in **decreasing order**:

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0$$

This ordering is crucial for PCA, as we select the top m eigenvectors corresponding to the largest eigenvalues.

Sample Covariance Matrix

Sample covariance matrix: Given data $\{x_i\}_{i=1}^N$ where $x_i \in \mathbb{R}^d$. For each dimension $j = 1, \ldots, d$, define:

$$\bar{x}_j = \frac{1}{N} \sum_{i=1}^{N} x_{ij}$$

Let $\bar{x} \in \mathbb{R}^d$ denote the vector of sample means with j-th coordinate $\bar{x}_j$.

Centering assumption: We assume $\bar{x} = 0$ (data is centered). This can always be achieved by replacing $x_i \leftarrow x_i - \bar{x}$ for all i.

For centered data, the sample covariance matrix is:

$$S = \frac{1}{N} \sum_{i=1}^{N} x_i x_i^T = \frac{1}{N} \mathbf{X}^T \mathbf{X}$$

Equivalently, element-wise: $S_{jk} = \frac{1}{N} \sum_{i=1}^{N} x_{ij} x_{ik}$ for $j, k = 1, \ldots, d$, where $\mathbf{X} \in \mathbb{R}^{N \times d}$ is the data matrix with i-th row equal to x_i^T.

*Some references use $1/(N-1)$ for unbiased covariance. We use $1/N$ normalization for simplicity; the scaling does not affect the eigenvectors.

Two Approaches to Dimensionality Reduction

Note: S is symmetric and PSD. We use S to denote sample covariance throughout.

❓ How should we project high-dimensional data onto a lower-dimensional subspace?

Two natural criteria:

1. **Maximize variance**: Choose the projection direction that preserves the most variation in the data
2. **Minimize reconstruction error**: Choose the projection that minimizes the distance between original and reconstructed data

Key insight: These two criteria are equivalent! Both lead to the same solution via eigenvectors of the covariance matrix S.

We now formalize these two approaches and prove their equivalence.

Two Formulations of PCA

Formulation 1: Maximum Variance. Find unit vector $u_1 \in \mathbb{R}^d$ such that projected data has maximum variance:

$$u_1 = \arg \max_{u:\|u\|=1} u^T S u$$

For m dimensions: find orthonormal $\{u_1, \ldots, u_m\}$ sequentially.

Formulation 2: Minimum Reconstruction Error. Find m-dimensional subspace minimizing average squared distance from data:

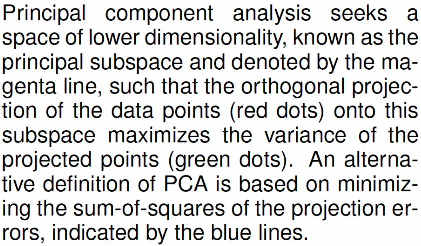
$$\min_{\{u_j\}_{j=1}^m} \frac{1}{N} \sum_{i=1}^{N} \left\| x_i - \sum_{j=1}^{m} (u_j^T x_i) u_j \right\|^2$$

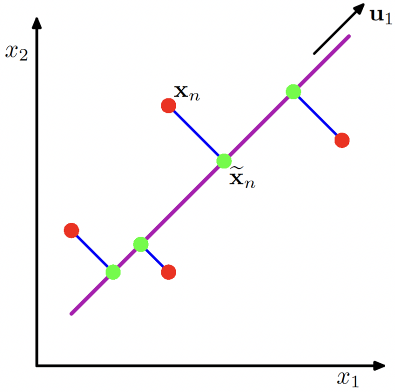
where $\{u_j\}$ are orthonormal.

PCA: Two Equivalent Formulations

Proposition 1

Both formulations yield the same solution: $\{u_1, \ldots, u_m\}$ are the top m eigenvectors of S (corresponding to largest eigenvalues).





Derivation: Maximum Variance Formulation

Setup: Let's start with the case $m = 1$, and seek a linear transformation from $\mathbb{R}^d$ to $\mathbb{R}$.

A general linear transformation of x_i to a scalar is:

$$z_i = u^T x_i, \quad u \in \mathbb{R}^d, \quad \|u\| = 1$$

(note: $\|u\| = 1$ ensures this corresponds to a projection map).

Goal: Choose u to maximize the sample variance of $\{z_i\}_{i=1}^N$.

Computing the variance: For centered data ($\bar{x} = 0$),

$$\begin{aligned}\mathrm{Var}(\{z_i\}) &= \frac{1}{N}\sum_{i=1}^N (z_i - \bar{z})^2 = \frac{1}{N}\sum_{i=1}^N z_i^2 \quad (\text{since } \bar{z} = u^T\bar{x} = 0) \\ &= \frac{1}{N}\sum_{i=1}^N (u^T x_i)^2 = \frac{1}{N}\sum_{i=1}^N u^T x_i x_i^T u = u^T S u,\end{aligned}$$

where we used $S = \frac{1}{N}\sum_{i=1}^N x_i x_i^T$.

Derivation: Maximum Variance (Complete)

Optimization problem[4]: $u_1 = \arg\max_{u \in \mathbb{R}^d : \|u\|=1} u^T S u$
Solution via Lagrange multipliers: Introduce Lagrangian

$$\mathcal{L}(u, \lambda) = u^T S u + \lambda(1 - u^T u)$$

First-order condition: $\frac{\partial \mathcal{L}}{\partial u} = 2Su - 2\lambda u = 0 \quad \Rightarrow \quad Su = \lambda u$
Thus u must be an eigenvector of S with eigenvalue λ.

Which eigenvector? If $Su = \lambda u$, then:

$$u^T S u = u^T (\lambda u) = \lambda (u^T u) = \lambda$$

The variance equals the eigenvalue! Maximum is achieved when $\lambda = \lambda_1$ (largest eigenvalue), with $u = u_1$ the corresponding eigenvector[5].

Higher dimensions: For $m > 1$, find u_2 maximizing variance subject to $\|u_2\| = 1$ and $u_2^T u_1 = 0$. This gives $u_2 =$ eigenvector corresponding to λ_2.

---

[4] Intuitively, clear from the interpretation of eigenvalues that u_1 should be the normalized eigenvector corresponding to the largest eigenvalue. Can also check this using the method of Lagrange multipliers.

[5] Hence, the projection map is $x \mapsto u_1^\top x$, where u_1 is the eigenvector corresponding to the largest eigenvalue λ_1. We call $\{z_i = u_1^\top x_i\}$ the *first PC scores* of the data x, with u_1 the *first PC axis*.

Derivation: Reconstruction Error Formulation

Goal: Find m-dimensional projection minimizing reconstruction error:

$$\min_{\{u_j\}_{j=1}^m} \frac{1}{N} \sum_{i=1}^{N} \left\| x_i - \sum_{j=1}^{m} (u_j^T x_i) u_j \right\|^2$$

subject to $u_j^T u_k = \delta_{jk}$ (orthonormality).

Setup: Consider an orthonormal basis $\{u_j : j = 1, \ldots, d\}$ for $\mathbb{R}^d$.
Any data point x_i can be represented as:

$$x_i = \sum_{j=1}^{d} \alpha_{ij} u_j = \sum_{j=1}^{d} (u_j^T x_i) u_j$$

where the second equality uses orthonormality: $\alpha_{ij} = u_j^T x_i$.

💡 m-**dimensional projection**: Project x_i onto a m-dim space spanned by the first m basis vectors so that:

$$z_i = \sum_{j=1}^{m} \beta_{ij} u_j$$

Derivation: Reconstruction Error (Incomplete)

- Our goal is to choose $\{\beta_{ij}\}$ and $\{u_j\}$ to minimize the reconstruction error

$$\frac{1}{N}\sum_{i=1}^{N}\|x_i - z_i\|^2, \text{ where } z_i = \sum_{j=1}^{m}\beta_{ij}u_j.$$

- We can show that (see blackboard) the minimum error is achieved when we choose $\{u_j\}$ to be the set of orthonormal eigenvectors of the covariance matrix S, ordered in decreasing eigenvalues.
- Consequently, the transformation that incurs the minimal error is given by the mapping $x \mapsto \sum_{j=1}^{m}(u_j^\top x)\, u_j$.
- Applying this transformation to each data point x_i, we obtain in matrix form the mapping $x \mapsto xU_m U_m^\top$, and the score matrix $Z_m = xU_m$ contains the coefficients (or PC scores) corresponding to the first m PCs.

Conclusion: Minimize error by choosing $\{u_{m+1}, \ldots, u_d\}$ with smallest eigenvalues. Equivalently, keep $\{u_1, \ldots, u_m\}$ with largest eigenvalues. Done with the proof for Proposition 1.

PCA Solution via Eigendecomposition

Given centered data with covariance $S = U\Lambda U^T$:

- The j-th principal component (PC) direction is u_j (eigenvector)
- The variance along j-th PC is λ_j (eigenvalue)
- Collect the first m PC scores into: $Z_m = \mathbf{X}U_m$ where $U_m = [u_1|\cdots|u_m] \in \mathbb{R}^{d\times m}$. The matrix $Z_m \in \mathbb{R}^{N\times m}$ has i-th row containing the scores of x_i on the first m PCs.
- Useful to look at **proportion of variance explained (PVE):** $\frac{\sum_{j=1}^m \lambda_j}{\sum_{j=1}^d \lambda_j}$

Interpretation:

- u_1 points in direction of greatest variance
- u_2 points in direction of greatest remaining variance (orthogonal to u_1)
- And so on...
- Eigenvectors with small λ correspond to "noise" directions

Reconstruction: The m-dimensional approximation is:

$$\mathbf{X}_m = Z_m U_m^T = \mathbf{X}U_m U_m^T$$

For individual points (centered case): $\hat{x}_i = \sum_{j=1}^m (u_j^T x_i)u_j = U_m U_m^T x_i$

For non-centered data: $\hat{x}_i = \bar{x} + U_m U_m^T(x_i - \bar{x})$

Principal Component Loadings and Scores (ISL's Notation)

Given centered data matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$ and its eigendecomposition:

$$S = \frac{1}{N}\mathbf{X}^\top \mathbf{X} = U \Lambda U^\top, \quad \Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_d), \quad U = [u_1 | \cdots | u_d].$$

- The **loading vectors** (columns of U) are

$$\phi_m = u_m, \quad m = 1, \ldots, d,$$

  where ϕ_{jm} measures the contribution of variable j to component m.
- The **scores** (principal component representations of the observations) are

$$z_{im} = x_i^\top u_m, \quad \text{or equivalently} \quad Z = \mathbf{X}U.$$

Reconstruction:

$$\begin{aligned} \hat{x}_i^{(M)} &= \sum_{m=1}^{M} z_{im}\phi_m = \sum_{m=1}^{M} (u_m^T x_i) u_m && \text{(for centered data)} \\ &= U_M U_M^T x_i && \text{(matrix form)} \end{aligned}$$

For non-centered data: $\hat{x}_i^{(M)} = \bar{x} + U_M U_M^T (x_i - \bar{x})$

where $\hat{x}_i^{(M)}$ is the best M-dimensional approximation of x_i in the PCA subspace.

The PCA Algorithm (In Full)

Principal Component Analysis (PCA)

1: **Input:** Data $\{x_i\}_{i=1}^N$ where $x_i \in \mathbb{R}^d$, dimension $m < d$
2: Form data matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$ with i-th row equal to x_i^T
3: **Center data:** Compute $\bar{x} = \frac{1}{N} \sum_{n=1}^N x_n$, then replace $x_i \leftarrow x_i - \bar{x} \ \forall i$
4: Compute sample covariance: $S = \frac{1}{N}\mathbf{X}^T\mathbf{X}$
5: Compute eigendecomposition: $S = U \Lambda U^T$
6: Extract top m eigenvectors: $U_m = [u_1 | \cdots | u_m]$ (corresponding to $\lambda_1 \geq \cdots \geq \lambda_m$)
7: Compute PC scores: $Z_m = \mathbf{X} U_m \in \mathbb{R}^{N \times m}$
8: **return** PC scores Z_m, loadings U_m, eigenvalues $\{\lambda_j\}_{j=1}^m$

Computational complexity:

- Full eigendecomposition of S: $O(d^3)$
- For $m \ll d$: Use power method / iterative methods: $O(md^2)$
- For $N \ll d$: Compute eigendecomposition of $\mathbf{X}\mathbf{X}^T$ instead ($N \times N$)
- In practice: Use SVD of $\mathbf{X}$ directly (more numerically stable)

PCA via Eigendecomposition vs. SVD

Singular Value Decomposition (SVD) approach: For a centered data matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$,

$$\mathbf{X} = UDV^\top,$$

where $U \in \mathbb{R}^{N \times N}$ and $V \in \mathbb{R}^{d \times d}$ are orthogonal, and $D \in \mathbb{R}^{N \times d}$ is diagonal (rectangular) with singular values $d_1 \geq \cdots \geq 0$.

Connection to PCA:

$$S = \frac{1}{N}\mathbf{X}^\top \mathbf{X} = \frac{1}{N} VD^\top DV^\top = \frac{1}{N} VD^2 V^\top.$$

Thus, principal component directions are the columns of V, and eigenvalues are $\lambda_j = d_j^2/N$.

Practice: Modern PCA computes the SVD of $\mathbf{X}$ directly — it is numerically more stable than forming and diagonalizing S.

*See Ch. 1 in `https://databookuw.com/databookV2.pdf` for a clear geometric introduction to the SVD.

Explained Variance by Each Principal Component

Definition 1

The **proportion of variance explained (PVE)** by the k-th principal component is

$$\mathrm{PVE}_k = \frac{\lambda_k}{\sum_{j=1}^{d} \lambda_j},$$

where λ_k is the k-th eigenvalue of the sample covariance matrix S.

Intuition: PVE quantifies how much of the total data variability is captured by each component.

💡 The total variance of the data can be decomposed as the variance captured by the first m principal components plus the mean squared reconstruction error of the best m-dimensional approximation (see also Exercise 14.2).

PVE as R^2 of the PCA Approximation

Proposition 2

For centered data ($\bar{x} = 0$), the PVE by the first m components is

$$\sum_{k=1}^{m} \mathrm{PVE}_k = 1 - \frac{\mathrm{RSS}}{\mathrm{TSS}},$$

where $\mathrm{TSS} := \sum_i \|x_i\|^2$ and $\mathrm{RSS} := \sum_i \|x_i - \hat{x}_i^{(m)}\|^2$.

Proof.
See Exercise 14.1. □

- TSS = total variance (total sum of squares)
- RSS = residual sum of squares after m-dimensional projection
- The ratio $1 - \mathrm{RSS}/\mathrm{TSS}$ is analogous to the R^2 **statistic** in regression: it measures the fraction of total variance captured by the model (here, by the PCA subspace).

Why Scaling the Variables

Because it is undesirable for the principal components obtained to depend on an arbitrary choice of scaling, we typically scale[6] each variable to have standard deviation one before we perform PCA.

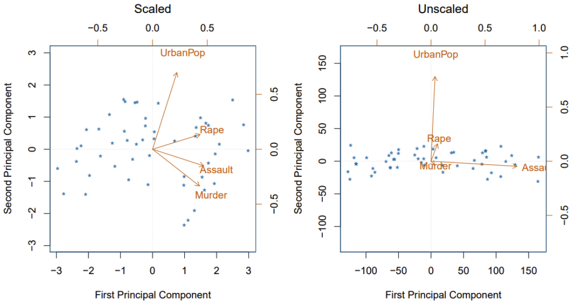


FIGURE 12.4. *Two principal component biplots for the* USArrests *data.* Left: *the same as Figure 12.1, with the variables scaled to have unit standard deviations.* Right: *principal components using unscaled data.* Assault *has by far the largest loading on the first principal component because it has the highest variance among the four variables. In general, scaling the variables to have standard deviation one is recommended.*

[6] In certain settings, however, the variables may be measured in the same units. In this case, we might not wish to scale the variables to have standard deviation one before performing PCA.

Choosing Number of Components

How to choose m?[7]

1. Scree plot: Plot eigenvalues $\lambda_1, \lambda_2, \ldots$ and look for "elbow" (a point at which the PVE by each subsequent principal component drops off)

2. Proportion of variance explained (PVE):

$$\mathrm{PVE}(m) = \frac{\sum_{j=1}^{m} \lambda_j}{\sum_{j=1}^{d} \lambda_j}$$

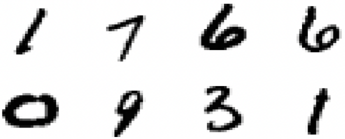
Common threshold: retain PCs explaining 80-90% of variance (signal is usually concentrated in its first few principal components)

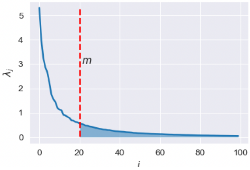
3. Cross-validation (CV): If we compute principal components for use in a supervised analysis, such as the principal components regression, then we can treat the number of principal component score vectors to be used in the regression as a tuning parameter to be selected via CV

[7] In general, the data matrix x has $\min(N-1, d)$ distinct PCs. However, we usually are not interested in all of them; rather, we would like to use just the first few PCs in order to visualize or interpret the data.

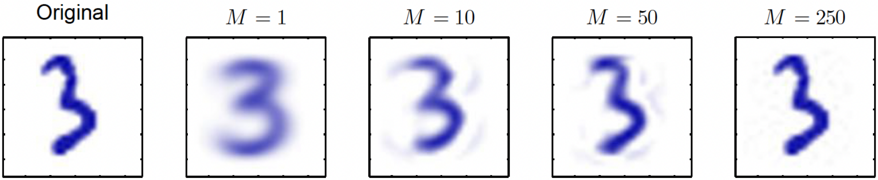
PCA on MNIST

Let us consider the MNIST dataset of hand-written digits. Performing a PCA on the dataset ($d = 784$) we obtain a eigenvalue distribution as shown in the following figure. Notice the decay of the eigenvalues of the sample covariance matrix (right plot). We can choose m to be sufficiently large so that the projection error $\sum_{j=m+1}^{d} \lambda_j$ (shaded area) is small enough.





PCA reconstructions obtained by retaining first M principal components for various values of M:



Applications of PCA

- Data visualization: Project to 2D or 3D for plotting
- Preprocessing: Remove correlations before supervised learning
- Compression: Reduce storage/transmission costs
- Denoising: Project to subspace, discard low-variance components
- Feature extraction: Use PCs as features in downstream tasks
- Principal component regression: Regress on PCs instead of original variables
- Imputation: Filling in the missing values in datasets (matrix completion, e.g., to power recommender systems that Netflix and Amazon use); see ISL Ch. 12.3 for details

Remark. PCA identifies orthogonal directions of maximum variance. A generalization known as **Independent Component Analysis (ICA)** aims to recover statistically independent sources, not just uncorrelated components (see Appendix 1).

PCA as Whitening/Sphering[8]

Transform the data so that its covariance becomes the identity matrix. Given eigendecomposition $S = U\Lambda U^\top$, define the whitening transform

$$x_i' = \Lambda^{-\frac{1}{2}} U^\top x_i, \quad \text{or equivalently (matrix form):} \quad X' = XU\Lambda^{-\frac{1}{2}}.$$

Assumption: For full whitening, $S \succ 0$ (all eigenvalues positive). If S is rank-deficient, use the pseudoinverse:

$$\Lambda^\dagger = \mathrm{diag}(\lambda_1^\dagger, \ldots, \lambda_d^\dagger), \quad \lambda_i^\dagger = \begin{cases} \lambda_i^{-1/2}, & \text{if } \lambda_i > 0, \\ 0, & \text{if } \lambda_i = 0. \end{cases}$$

Then the whitened data satisfy:

$$\frac{1}{N}\sum_i x_i' = 0, \qquad \frac{1}{N}\sum_i x_i'(x_i')^\top = \Lambda^{-\frac{1}{2}} U^\top S U \Lambda^{-\frac{1}{2}} = I.$$

Whitening removes all correlations (features become uncorrelated and unit-variance), but the components are not necessarily statistically independent — this motivates **ICA**.

[8] Useful as a preprocessing step for algorithms that assume uncorrelated features, e.g., Independent Component Analysis (ICA).

Ridge Regression in Principal Components Space

Principal component (PC) regression with ridge regularization performs selective shrinkage along PCs

Let $\mathbf{X} = U\Sigma V^T$ be the SVD of $\mathbf{X} \in \mathbb{R}^{N\times d}$. Then ridge fitted values are:

$$\begin{aligned}\mathbf{X}\hat{\beta}_{ridge} &= \mathbf{X}(\mathbf{X}^T\mathbf{X} + \lambda I)^{-1}\mathbf{X}^T y \\ &= U\Sigma^2(\Sigma^2 + \lambda I)^{-1}U^T y \\ &= \sum_{j=1}^{\min(N,d)} \frac{\sigma_j^2}{\sigma_j^2 + \lambda} u_j u_j^T y\end{aligned}$$

where u_j is the j-th column of U, and σ_j is the j-th singular value.

Comparison with least squares:
Least squares projection: $\mathbf{X}\hat{\beta}_{LS} = UU^T y = \sum_{j=1}^{\min(N,d)} u_j u_j^T y$

Key insight: Ridge performs *non-uniform* shrinkage:

- Shrinks more along directions u_j with **small** σ_j (low variance)
- Shrinks less along directions u_j with **large** σ_j (high variance)
- Shrinkage factor: $\frac{\sigma_j^2}{\sigma_j^2+\lambda} \in (0, 1)$ for $\lambda > 0$

Probabilistic PCA (PPCA): Motivation

Question: Can we give the principal components a probabilistic interpretation?

Benefits of probabilistic formulation:

- Handle missing data naturally
- Quantify uncertainty in latent representations
- Compare models via likelihood
- Enable Bayesian extensions (priors on parameters)
- Derive EM algorithm for fitting

Key idea: Model data as generated from latent variables via linear transformation plus Gaussian noise. This is the idea behind Probabilistic PCA[9] (PPCA). The framework of PPCA also allows us to extend the method and alter its underlying assumptions easily.

[9] Tipping-Bishop (1998); `jstor.org/stable/2680726`.

Probabilistic PCA: Model Definition

Definition 2: Probabilistic PCA Model

Latent variable model with:

$$z \sim \mathcal{N}(0, I_m) \quad \text{(latent variables)}$$
$$x|z \sim \mathcal{N}(Wz + \mu, \sigma^2 I_d) \quad \text{(observations)}$$

- $z \in \mathbb{R}^m$ is latent (principal component scores)
- $W \in \mathbb{R}^{d \times m}$ is loading matrix
- $\mu \in \mathbb{R}^d$ is mean
- σ^2 is isotropic noise variance

Next, we will derive:

- **Marginal distribution**: $p(x) = \mathcal{N}(\mu, C)$ where $C = WW^T + \sigma^2 I_d$
- **Posterior**: $p(z|x) = \mathcal{N}(M^{-1}W^T(x - \mu), \sigma^2 M^{-1})$, where $M = W^T W + \sigma^2 I_m$

Probabilistic PCA: Marginal Distribution

Model: $z \sim \mathcal{N}(0, I_m), \quad x \mid z \sim \mathcal{N}(Wz + \mu, \sigma^2 I_d).$

- Write $x = \mu + Wz + \varepsilon$, where $\varepsilon \sim \mathcal{N}(0, \sigma^2 I_d)$ and $\varepsilon \perp z$.
- Then

$$\mathbb{E}[x] = \mu, \qquad \mathrm{Cov}(x) = W\mathbb{E}[zz^\top]W^\top + \mathrm{Cov}(\varepsilon) = WW^\top + \sigma^2 I_d.$$

- Hence the marginal distribution is

$$p(x) = \mathcal{N}(\mu, C), \quad \text{with } C = WW^\top + \sigma^2 I_d.$$

Intuition:

The data lie near a linear subspace spanned by the columns of W, with isotropic Gaussian noise of variance σ^2. The standard prior $z \sim \mathcal{N}(0, I_m)$ is without loss of generality (see Exercise 14.3).

Probabilistic PCA: Posterior Derivation

Goal: derive $p(z \mid x)$ for $z \sim \mathcal{N}(0, I_m)$, $x \mid z \sim \mathcal{N}(Wz + \mu, \sigma^2 I_d)$.

$$\begin{aligned}\log p(z \mid x) &= \log p(z) + \log p(x \mid z) + \text{const} \\ &= -\tfrac{1}{2} z^\top z - \tfrac{1}{2\sigma^2} \|x - \mu - Wz\|^2 + \text{const.}\end{aligned}$$

- Expand the quadratic term:

$$\|x - \mu - Wz\|^2 = (x - \mu)^\top (x - \mu) - 2z^\top W^\top (x - \mu) + z^\top W^\top W z.$$

- Collect terms in z:

$$\log p(z \mid x) = -\tfrac{1}{2} z^\top \Big(I_m + \tfrac{1}{\sigma^2} W^\top W \Big) z + \tfrac{1}{\sigma^2} z^\top W^\top (x - \mu) + \text{const.}$$

- Define $Q^{-1} = I_m + \frac{1}{\sigma^2} W^\top W$ and $\eta = \frac{1}{\sigma^2} W^\top (x - \mu)$.
- Completing the square gives

$$p(z \mid x) = \mathcal{N}(Q\eta, Q),$$

  where $Q = (I_m + \frac{1}{\sigma^2} W^\top W)^{-1} = \sigma^2 (W^\top W + \sigma^2 I_m)^{-1} = \sigma^2 M^{-1}$.
- Thus, $p(z \mid x) = \mathcal{N}\big(M^{-1} W^\top (x - \mu),\, \sigma^2 M^{-1}\big)$, with $M = W^\top W + \sigma^2 I_m$.

PPCA and Bayesian Linear Regression

The PPCA model $z \sim \mathcal{N}(0, I_m)$, $x \mid z \sim \mathcal{N}(Wz + \mu, \sigma^2 I_d)$ can be viewed as a *Bayesian linear regression model* in the variable z.

- In Bayesian linear regression, $y = A\theta + \varepsilon$, with $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$, $\theta \sim \mathcal{N}(0, I)$.
- The posterior is $p(\theta \mid y) = \mathcal{N}\big((A^\top A + \sigma^2 I)^{-1} A^\top y,\ \sigma^2 (A^\top A + \sigma^2 I)^{-1}\big)$.

PPCA as Bayesian regression (transposed view):

- Each observed dimension x_j is a linear regression of z: $x_j = w_j^\top z + \mu_j + \varepsilon_j$, with $\varepsilon_j \sim \mathcal{N}(0, \sigma^2)$.
- For fixed W, each row $w_j^\top$ acts as regression weights, and z acts as the *input*.
- The posterior over z given x matches the Bayesian regression posterior: $p(z \mid x) = \mathcal{N}\big(M^{-1} W^\top (x - \mu),\ \sigma^2 M^{-1}\big)$ where $M = W^\top W + \sigma^2 I_m$.

Interpretation:

- PPCA $\Rightarrow$ Bayesian regression *in the latent space*.
- The latent variable z plays the role of regression weights θ.
- The mapping $x \approx Wz + \mu$ defines a probabilistic low-dimensional subspace.

Bayesian Linear Regression as Transposed PPCA

Bayesian linear regression model: $\theta \sim \mathcal{N}(0, I_m), \quad y \mid \theta \sim \mathcal{N}(A\theta + b, \sigma^2 I_n)$.

- Posterior over θ:

$$p(\theta \mid y) = \mathcal{N}\big((A^\top A + \sigma^2 I_m)^{-1} A^\top (y - b),\, \sigma^2 (A^\top A + \sigma^2 I_m)^{-1}\big).$$

- This has the same algebraic form as the PPCA posterior $p(z \mid x) = \mathcal{N}\big(M^{-1} W^\top (x - \mu),\, \sigma^2 M^{-1}\big)$, with $M = W^\top W + \sigma^2 I_m$.

Transposed correspondence:

Bayesian regression		Probabilistic PCA
$y = A\theta + b + \varepsilon$	$\leftrightarrow$	$x = Wz + \mu + \varepsilon$
θ (parameters)	$\leftrightarrow$	z (latent variables)
$A^\top A + \sigma^2 I_m$	$\leftrightarrow$	$W^\top W + \sigma^2 I_m$

Interpretation:

- PPCA is the *dual* of Bayesian linear regression.
- In regression, we infer θ (weights) given observed y.
- In PPCA, we infer z (latent coordinates) given observed x.
- Both are linear–Gaussian models with identical posterior structure.

Summary: Three Views of Probabilistic PCA

1. Model (Generative View)

- Latent variable model: $z \sim \mathcal{N}(0, I_m), \quad x \mid z \sim \mathcal{N}(Wz + \mu, \sigma^2 I_d)$.
- Marginal: $p(x) = \mathcal{N}(\mu, WW^\top + \sigma^2 I_d)$.

2. Inference (Posterior View)

- Posterior over latent variables: $p(z \mid x) = \mathcal{N}\big(M^{-1} W^\top (x - \mu),\, \sigma^2 M^{-1}\big)$ with $M = W^\top W + \sigma^2 I_m$.

3. Duality (Bayesian Regression View)

- PPCA is mathematically equivalent to Bayesian linear regression, with roles of parameters and inputs swapped.
- Regression: infer θ from $y = A\theta + \varepsilon$. PPCA: infer z from $x = Wz + \varepsilon$.
- Both yield Gaussian posteriors with identical algebraic form.

PPCA unifies linear projection, probabilistic inference, and Bayesian linear regression.

PPCA: Visual Illustration

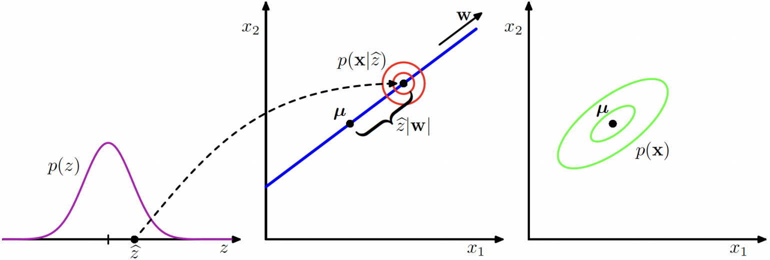


Figure 16.7 An illustration of the generative view of a probabilistic PCA model for a two-dimensional data space and a one-dimensional latent space. An observed data point $\mathbf{x}$ is generated by first drawing a value $\hat{z}$ for the latent variable from its prior distribution $p(z)$ and then drawing a value for $\mathbf{x}$ from an isotropic Gaussian distribution (illustrated by the red circles) having mean $\mathbf{w}\hat{z} + \boldsymbol{\mu}$ and covariance $\sigma^2\mathbf{I}$. The green ellipses show the density contours for the marginal distribution $p(\mathbf{x})$.

We can use the EM[10] that we learned in Lecture 13 to find the MLE of parameters $\{W, \mu, \sigma^2\}$ given $\{x_i\}_{i=1}^N$. We will not derive the algorithm (see PRML Ch. 12.2.2).

*PPCA is a special case of the Factor Analysis model where the conditional distribution of x given z has an **isotropic** (rather than diagonal) covariance matrix.

[10] For standard PPCA with moderate dataset sizes, EM is typically faster and more reliable than SGD.

PPCA vs Standard PCA

Denote the **MLE solution**: $W := W_{MLE} = U_m(\Lambda_m - \sigma^2 I)^{1/2} R$

- U_m contains top m eigenvectors of sample covariance S
- $\Lambda_m = \mathrm{diag}(\lambda_1, \ldots, \lambda_m)$ are eigenvalues
- R is arbitrary $m \times m$ rotation matrix
- $\sigma^2 = \frac{1}{d-m} \sum_{j=m+1}^{d} \lambda_j$ (avg of discarded eigenvalues)

Limit behavior as $\sigma^2 \to 0$:

- When noise vanishes, $W \to U_m \Lambda_m^{1/2} R$
- The columns of W span the same subspace as U_m (principal subspace)
- Reconstruction: $\hat{x} = \mu + W(W^\top W)^{-1} W^\top (x - \mu) = \mu + U_m U_m^\top (x - \mu)$
- This is *exactly* the PCA projection onto the top m components
- See Exercise 14.4: Shows this reconstruction holds for *any* $\sigma^2 \geq 0$!

Comparison:

Property	**PCA**	**PPCA**
Framework	Deterministic	Probabilistic
Handles missing data	No	Yes (via EM)
Uncertainty	No	Yes
Noise model	Implicit	Explicit (σ^2)
Model selection	Ad hoc	Likelihood-based

Nonlinear Extensions of PCA

So far: Linear dimensionality reduction

- PCA: deterministic, maximizes variance
- PPCA: probabilistic, explicit noise model
- Both[11]: $x \approx Wz$ (linear relationship)

? What if relationships are nonlinear?

Two approaches:

1. **Kernel PCA**: Apply PCA in feature space
2. **Autoencoders (AEs)**: Extend PCA (linear AE) to nonlinear version using neural networks

[11] Try the experimental exercises to better understand PCA and PPCA.

Kernel PCA: PCA in Feature Space

Limitation of standard PCA: Only finds linear structure

Solution: Apply PCA after nonlinear feature mapping $\phi : \mathbb{R}^d \to \mathbb{R}^M$

Kernel PCA

1: Choose feature map ϕ (often implicitly via kernel)
2: Form design matrix $\Phi \in \mathbb{R}^{N \times M}$ where $\Phi_{ij} = \phi_j(x_i)$
3: Center: $\Phi_{ij} \leftarrow \Phi_{ij} - \frac{1}{N} \sum_k \Phi_{kj}$
4: Compute covariance in feature space: $S_\phi = \frac{1}{N} \Phi^T \Phi$
5: Eigendecomposition: $S_\phi = U \Lambda U^T$
6: PC scores in feature space: $Z_m = \Phi U_m$
7: **return** PC scores Z_m, loadings U_m, eigenvalues $\{\lambda_j\}_{j=1}^m$

Kernel trick: Can formulate using kernel $k(x, x') = \phi(x)^T \phi(x')$

- Polynomial: $k(x, x') = (1 + x^T x')^p$
- Gaussian RBF: $k(x, x') = \exp(-\|x - x'\|^2 / 2\sigma^2)$ (allows using features living on an infinite-dimensional space)

Autoencoders (AEs): Nonlinear Extension of PCA

PCA as lossy encoding-decoding: Recall the principal component scores are given by:

$$Z = \mathbf{X}U \in \mathbb{R}^{N \times d}$$

Since U is orthogonal, we can invert this and get: $\mathbf{X} = ZU^T$.

Dimensionality reduction: Use only first m columns of U:

- **Encoding**: $Z_m = \mathbf{X}U_m \in \mathbb{R}^{N \times m}$ (compress to m dimensions)
- **Decoding**: $\mathbf{X}_m = Z_m U_m^T \in \mathbb{R}^{N \times d}$ (approximate reconstruction)

Key observation: If $m < d$, then generally $\mathbf{X}_m \neq \mathbf{X}$ (lossy compression).

The reconstruction error is $\sum_{j=m+1}^{d} \lambda_j$ (sum of discarded eigenvalues).

💡 The low-dimensional representation Z_m of $\mathbf{X}$ is called the **latent representation** or simply **latents**. Each row of Z_m is a reduced representation of the corresponding row of $\mathbf{X}$. In this case, the latent space has dimension m.

PCA: Encoding and Decoding Maps

Latent space: The space $\mathbb{R}^m$ where $m < d$ is called the latent space.

Encoding map $T_{enc} : \mathbb{R}^d \to \mathbb{R}^m$:
For each input $x \in \mathbb{R}^d$:

$$T_{enc}(x)_j = x^T u_j = (U_m^T x)_j, \quad j = 1, \ldots, m$$

In vector form: $z = T_{enc}(x) = U_m^T x \in \mathbb{R}^m$

Decoding map $T_{dec} : \mathbb{R}^m \to \mathbb{R}^d$:
For each latent $z \in \mathbb{R}^m$:

$$T_{dec}(z)_j = (U_m z)_j, \quad j = 1, \ldots, d$$

In vector form: $\hat{x} = T_{dec}(z) = U_m z \in \mathbb{R}^d$

Full encoding-decoding:

$$x \xrightarrow{T_{enc}} z = U_m^T x \xrightarrow{T_{dec}} \hat{x} = U_m z = U_m U_m^T x$$

⚠ These are linear transformations and may not be sufficient in finding good/efficient latent representations. Can we generalize to nonlinear?

Autoencoders: Nonlinear Generalization

Key idea: Replace the linear maps in PCA with nonlinear parameterized functions (e.g., neural networks).

Definition 3: Autoencoder

Consider two general parameterized mappings:

- **Encoder**: $T_{enc}(x;\theta) : \mathbb{R}^d \to \mathbb{R}^m$ maps to latent code
- **Decoder**: $T_{dec}(z;\phi) : \mathbb{R}^m \to \mathbb{R}^d$ reconstructs from latent

where θ and ϕ are adjustable parameters (to be learned).
Typically $m < d$ (bottleneck forces compression).

Training objective: Minimize reconstruction error:

$$\min_{\theta,\phi} \frac{1}{N} \sum_{i=1}^{N} \|x_i - T_{dec}(T_{enc}(x_i;\theta);\phi)\|^2$$

Use SGD with backpropagation to train both θ and ϕ jointly.

Autoencoders: Properties and Applications

Advantages over PCA:

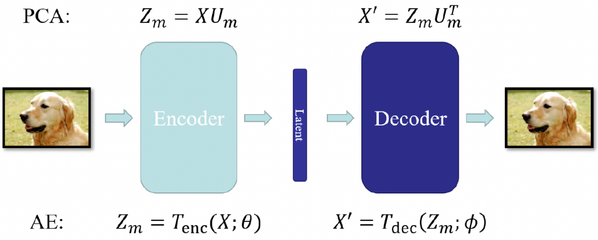
- Can capture **nonlinear structure** in data
- More flexible representation learning
- Can use deep architectures for hierarchical features

Applications:

- Dimensionality reduction for visualization
- Feature learning for downstream tasks
- Denoising (denoising autoencoders)
- Data compression
- Anomaly detection (high reconstruction error $\Rightarrow$ anomaly)
- Pretraining for supervised learning

Note: A linear AE recovers the same subspace as PCA — i.e., it identifies the same directions of variation. But because it doesn't constrain its encoder/decoder weights to be orthogonal, the specific basis vectors it learns can differ (by any invertible linear transformation) from PCA's eigenvector basis.

Autoencoders: Intuition and Visualization



The bottleneck layer (latent space with dimension $m < d$) forces dimensionality reduction.

Both PCA and autoencoders can be viewed as compression-decompression algorithms involving latent representations!

Probabilistic Extensions: Variational AEs (VAEs) are a nonlinear probabilistic generalization of PPCA that replaces linear-Gaussian mappings with nonlinear ones, and replaces exact inference with approximate inference.

Variational AEs (VAEs): Probabilistic Extension[12]

Ordinary AEs learn a deterministic mapping $x \mapsto z \mapsto \hat{x}$. But the latent space z is often irregular — many z values do not decode to meaningful samples.

Key idea: Make the AE **probabilistic** by introducing a latent variable model:

$$p_\phi(x, z) = p_\phi(x|z)\, p(z),$$

where $p(z) = \mathcal{N}(0, I)$ and $p_\phi(x|z)$ is represented by a neural decoder network.

Encoder as inference: Instead of outputting a point, the encoder produces a distribution

$$q_\theta(z|x) = \mathcal{N}(z;\, \mu_\theta(x),\, \Sigma_\theta(x))$$

that approximates the true posterior $p_\phi(z|x)$.

Training objective: Maximize a lower bound on $\log p_\phi(x)$ (called the **ELBO**)

💡 VAEs combine deep generative modeling with approximate Bayesian inference, generalizing both **PPCA** and **AEs**.

[12] This is an optional topic and we only give a high-level intro; see Ch. 19 in Bishop (`https://www.bishopbook.com/`) if you are interested in the omitted details.

Summary of Unsupervised Learning: A Unified View

I. Dimensionality Reduction Methods

Method	**Type**	**Prob[13].?**	**Comments**
PCA	Linear	No	Orthogonal projection; maximizes variance
PPCA	Linear	Yes	Linear-Gaussian latent variable model
ICA	Linear	Yes	Independent non-Gaussian sources
Kernel PCA	Nonlinear	No	Implicit nonlinear mapping via kernels
AE	Nonlinear	No	Deterministic neural encoder-decoder mapping
VAE	Nonlinear	Yes	Probabilistic nonlinear latent-variable model

II. Broader Perspective: Latent Variable Modeling Across Tasks

- **Clustering and Mixture Models:** K-means (hard) $\leftrightarrow$ GMM (soft, probabilistic); GMM with $\sigma^2 \to 0$ recovers K-means.
- **Dimensionality Reduction and Beyond:** PCA $\to$ PPCA $\to$ Autoencoder $\to$ VAE (linear $\to$ nonlinear, deterministic $\to$ probabilistic).

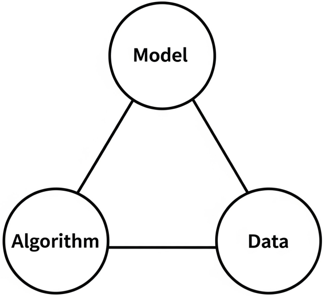
💡 Unsupervised learning is exploratory; latent variables explain observed structure and variation in data.

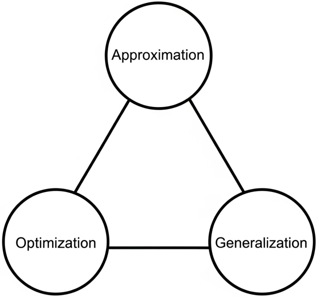
[13] Short hand for "Probabilistic?"

The Full Circle: Our Story of Learning from Data

We saw this slide on our first lecture:

Supervised Learning	**Unsupervised Learning**
Regression	Clustering
Classification	Dimensionality reduction
Function approximation	Distribution learning





Since then, we have enriched our understanding, connecting tasks to a powerful set of methods and principles!

The Full Circle: Our Story of Learning from Data

Supervised
(Lectures 1–12)

- **Linear methods**: regression (L2), classification (L3), SVCs/SVMs (L4)
- **Beyond linearity**: basis expansion (L5), local averaging, kernel smoothing (L5), kernel methods (L6–7)
- **Model selection** & generalization (L8)
- **Ensembles**: trees, forests, boosting (L9)
- **Neural networks:** deep learning (L10) & modern phenomena (L11)
- **Reliability**: Bayesian approach (L12)
- **Not covered**: deep learning for structured data, many others

Learning from Data

Unsupervised
(Lectures 13–14)

- **Clustering**: K-means, GMMs (L13)
- **Dim reduction**: PCA, PPCA, kernel PCA, autoencoders (L14)
- **Not covered**: ICA, VAEs, many others

Unifying Framework: The Learning Triad

Model (expressiveness) $\leftrightarrow$ **Algorithm** (optimization) $\leftrightarrow$ **Data** (generalization)

All ML methods—supervised or unsupervised—involve balancing model complexity, algorithmic efficiency, and generalization. Understanding this triad is essential for principled machine learning.

Exercise: Explained Variance in PCA

Exercise 14

1. Let $\{x_i\}_{i=1}^N$ be centered data with covariance $S = \frac{1}{N}\sum_i x_i x_i^\top$ and eigendecomposition $S = U\Lambda U^\top$, where $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_d)$ with $\lambda_1 \geq \cdots \geq \lambda_d \geq 0$.
   (a) Show that the total variance equals $\mathrm{tr}(S) = \sum_{j=1}^d \lambda_j$, and that the variance captured by the first m principal components is $\sum_{j=1}^m \lambda_j$.
   (b) Define the proportion of variance explained (PVE) by the first m components as
   $$\mathrm{PVE}(m) := \frac{\sum_{j=1}^m \lambda_j}{\sum_{j=1}^d \lambda_j}.$$
   Show that both $\mathrm{tr}(S)$ and $\mathrm{PVE}(m)$ are invariant under orthogonal coordinate transformations.
   (c) (**PVE as R^2**) Let $U_m = [u_1, \ldots, u_m]$ and $\hat{x}_i^{(m)} = U_m U_m^\top x_i$. Show that
   $$\mathrm{PVE}(m) = 1 - \frac{\sum_{i=1}^N \|x_i - \hat{x}_i^{(m)}\|^2}{\sum_{i=1}^N \|x_i\|^2}.$$

Exercise: Eigenvector Characterization of PCA

Exercise 14

2. Show by induction that the M-dimensional subspace maximizing projected variance is spanned by the M eigenvectors of S with the largest eigenvalues.
   (a) **Base case:** For $M = 1$, show that the direction U_1 maximizing $U_1^\top SU_1$ with $\|U_1\| = 1$ satisfies $SU_1 = \lambda_1 U_1$.
   (b) **Inductive step:** Assume the claim holds for M orthonormal eigenvectors $U_1, \ldots, U_M$. Introduce U_{M+1} orthogonal to the first M and normalized to unit length.
   (c) Form the Lagrangian enforcing the constraints and derive the first-order condition. Show that it implies $SU_{M+1} =_{M+1}$ for some scalar α.
   (d) Argue that the variance along U_{M+1} is maximized when $\alpha = \lambda_{M+1}$, the next largest eigenvalue.

   By induction, the subspace maximizing total projected variance is spanned by the top M eigenvectors of S.

Exercise: PPCA Reparameterization Invariance

Exercise 14

3. Consider the probabilistic PCA model with conditional $p(x \mid Z) = \mathcal{N}(x \mid WZ + \mu, \sigma^2 I)$. Replace the standard latent prior $Z \sim \mathcal{N}(0, I)$ by a general Gaussian prior $Z \sim \mathcal{N}(M, \Sigma_z)$ with $\Sigma_z \succ 0$.

   (a) Derive the marginal over the observed variables and show that it remains Gaussian.

   (b) Let $\Sigma_z = LL^\top$ for some invertible L, and define $Z' = L^{-1}(Z - M)$. Express $p(x')$ in the same form as the PPCA conditional $p(x') = \mathcal{N}(W'Z' + \mu', \sigma^2 I)$, identifying the corresponding (W', μ').

   (c) Conclude that the family of marginals $p(x)$ generated by any Gaussian latent prior is identical (up to reparameterization) to that obtained under the standard PPCA prior $\mathcal{N}(0, I)$.

Exercise: PPCA Reconstruction Equals PCA Projection

Exercise 14

4. In probabilistic PCA, $x \mid Z \sim \mathcal{N}(WZ + \mu, \sigma^2 I)$ with latent $Z \sim \mathcal{N}(0, I)$. Show that the optimal reconstruction of x according to the conventional PCA least–squares criterion is $\hat{x} = \mu + U_m U_m^\top (x - \mu)$.
   (a) Derive the expression for $Z^\star$ that minimizes $\|x - \mu - WZ\|^2$.
   (b) Substitute $Z^\star$ to express the reconstructed point $\hat{x}$ and interpret the resulting operator geometrically.
   (c) Using the PPCA MLE form $W = U_m(\Lambda_m - \sigma^2 I)^{1/2} R$, show that the reconstruction operator reduces to $U_m U_m^\top$. Hence, the PPCA reconstruction coincides with the standard PCA projection.
5. **[Experimental]**
   (a) **Two Approaches to Compute PVE.** Solve Exercise 12.6.8 in ISL.
   (b) **PCA and PPCA on the MNIST Task.** Work through: `https://colab.research.google.com/github/uu-sml/course-apml-public/blob/master/exercises/Session11.ipynb` and `https://colab.research.google.com/github/uu-sml/course-apml-public/blob/master/lab/PPCA.ipynb`.

Appendix: Advanced Topics

We have covered only the most basic methods. What interesting unsupervised learning methods should you explore next? Here is a preview:

- **Independent component analysis (ICA):** Generalization of PCA that seeks statistically independent, typically non-Gaussian, latent sources.
- **Modern visualization and representation techniques:** t-SNE, UMAP, NMF, etc.
- **Generative modeling:** learning probability distributions to synthesize new data

These methods extend classical unsupervised learning toward high-dimensional representation, synthesis, and reasoning.

Independent Component Analysis (ICA)[14]

Motivation: PCA finds uncorrelated components that maximize variance. ICA goes further to find *statistically independent* components.

The ICA Model:

- Observed data: $x = AS$ where $x \in \mathbb{R}^d$ (observations)
- $S \in \mathbb{R}^d$ are *latent sources* (statistically independent)
- $A \in \mathbb{R}^{d \times d}$ is unknown mixing matrix (square, invertible)
- Goal: Find unmixing matrix $W = A^{-1}$ so that $\hat{S} = Wx$ recovers independent sources

Example (Cocktail Party Problem): Multiple microphones record multiple speakers simultaneously. Want to separate individual voices from mixed signals.

Key Difference from PCA:

- PCA: Finds orthogonal transformation, uncorrelated components
- ICA: Finds general linear transformation, independent components

[14] See PRML Ch. 12.4.

ICA: Key Assumptions and Identifiability

Fundamental Assumptions:

1. Sources $s_1, \ldots, s_d$ are *statistically independent*: $p(S) = \prod_{i=1}^{d} p(s_i)$
2. At most one source can be Gaussian
3. Mixing matrix A is square and invertible

Why "at most one Gaussian"?
If all sources were independent Gaussians:

$$p(S) = \prod_i \mathcal{N}(s_i; 0, 1) = \mathcal{N}(S; 0, I)$$

Then observed data: $p(x) = \mathcal{N}(0, AA^T)$
Problem: A is only identifiable up to orthogonal rotation (any $A' = AR$ where $R^T R = I$ gives same covariance). Non-Gaussianity breaks this symmetry.

Ambiguities that remain even with non-Gaussian sources:

- Order of components (can permute sources)
- Scale of components (can multiply s_i by constant, divide A_i by same)
- Sign of components (can flip sign of both)

ICA: Exploiting Non-Gaussianity

Central Idea: By the Central Limit Theorem, sums of independent random variables are *more Gaussian* than the original variables.

Since $x = AS$ represents mixtures (linear combinations) of sources, the observations are more Gaussian than the sources.

$\implies$ **To unmix**: Find directions that are *maximally non-Gaussian*.

Measures of Non-Gaussianity:

1. Kurtosis:

$$\text{kurt}(y) = \frac{\mathbb{E}[y^4]}{(\mathbb{E}[y^2])^2} - 3$$

For standardized variable (unit variance): $\text{kurt}(y) = \mathbb{E}[y^4] - 3$

- $\text{kurt}(y) = 0$ for Gaussian
- $\text{kurt}(y) > 0$ for super-Gaussian (heavy-tailed)
- $\text{kurt}(y) < 0$ for sub-Gaussian (light-tailed)
- Simple to compute but sensitive to outliers

ICA: Negentropy and Maximum Likelihood

2. Negentropy:
Among all distributions with given variance, Gaussian has maximum entropy. Define:

$$J(y) = H(y_{\text{gauss}}) - H(y)$$

where $H(y) = -\int p(y) \ln p(y) dy$ is differential entropy, and y_{gauss} is Gaussian with same mean and variance as y.

Properties:

- $J(y) \geq 0$ always, with equality iff y is Gaussian
- Invariant under linear transformation
- Difficult to compute directly

Approximation:

$$J(y) \approx \sum_{i=1}^{p} k_i \{\mathbb{E}[G_i(y)] - \mathbb{E}[G_i(\nu)]\}^2$$

where $\nu \sim \mathcal{N}(0,1)$, k_i are positive constants, and G_i are nonquadratic functions.

Common choice: $G(u) = \frac{1}{a} \log \cosh(au)$ for $a \in [1,2]$

ICA: Estimation and FastICA Algorithm

Maximum Likelihood Approach: Given observations $\{x_n\}$, assume sources have densities $p(s_i)$. The log-likelihood:

$$\ln p(x|W) = \sum_{n=1}^{N} \left\{ \sum_{i=1}^{d} \ln p_i({w_i}^T x_n) + \ln|\det(W)| \right\}$$

where $W = A^{-1}$ is the unmixing matrix.

Standard Preprocessing:

1. **Center the data**: Subtract mean
2. **Whiten the data**: Use PCA to transform to unit covariance

$$\tilde{x} = U\Lambda^{-1/2}U^T(x - \bar{x})$$

where U^T is eigendecomposition of covariance

After whitening, only need to find orthogonal rotation matrix.

FastICA: Efficient fixed-point algorithm that maximizes non-Gaussianity (negentropy approximation) using Newton-like iterations. Computationally fast and widely used.

Modern Visualization & Representation Methods

Visualization and Embedding Methods

- **t-SNE, UMAP**: Nonlinear embedding techniques that reveal clusters and manifold structure in high-dimensional data.
- **PCA, MDS**: Classical linear methods preserving global Euclidean relationships.

Matrix and Manifold Learning

- **NMF**: Nonnegative Matrix Factorization for interpretable, parts-based representations.
- **SOMs, Isomap, LLE**: Neural and geometric approaches for discovering nonlinear manifolds.

These methods (see ESL Ch. 14) help us **visualize**, **compress**, and **interpret** high-dimensional data – laying the foundation for modern **representation learning** and **generative modeling**.

Modern Generative Modeling

Generative Models[15] learn a data distribution to create realistic new samples:

- **VAEs**: Probabilistic autoencoders with structured latent spaces.
- **GANs**: Adversarial training between generator and discriminator for photorealistic synthesis.
- **Normalizing Flows**: Invertible neural mappings with exact, tractable likelihoods.
- **Diffusion Models**: Learn to reverse a gradual noising process for stable, high-quality generation.

Applications: Image, text, and audio generation; and scientific applications.

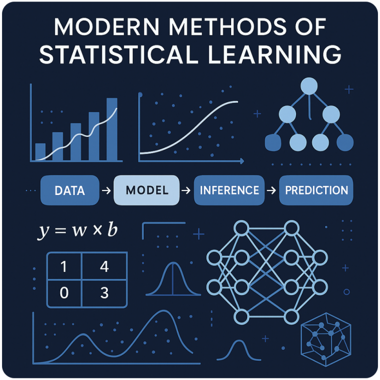
Today's frontier: Diffusion-based models power AI image and video generation tools such as **ChatGPT (DALL-E 3)**, **Stable Diffusion**, **Google Veo**. Yet, the foundations behind them are still evolving, with many open questions to explore!

[15] See `https://link.springer.com/book/10.1007/978-3-031-64087-2` for a deep dive into deep generative models, or see Ch. 17-20 in Bishop's modern version of PRML for a solid intro.

The Learning Journey Continues...

Generate a visually engaging illustration for a master's-level course titled "Modern Methods of Statistical Learning."

Image created



Thank you for being part of this course.
Wishing you success in applying what you've learned!